SECTION B · SKILL TAXONOMY · SPRINT 3 — REPRESENTATION & CLUSTERING

Sprint 3 report: from skill statements to leaf topics

Northeastern "Taxonomy of Work" capstone · status 2026-10-22 (night) · written for the team, as the source for the demo slides. Numbers come from the repo (`data/processed/sprint3_full_2026-10-22/canonical_v2/` , Supabase, `docs/HANDOFF.md`).

1. From Sprint 2 2. What Sprint 3 is and how we did it 3. Clustering methods and results 4. Skill library 5. Parameter sweep 6. Final result Appendix: reproduce

11,093	374	0.84	196
clean skills (from 11,449 names)	leaf topics (HDBSCAN)	stability AMI (80% subsets)	library skills · 249 strong links

1. How we progressed from Sprint 2

Sprint 2 (Text Extraction) turned each posting into atomic skill statements and measured their quality. It ended with:

Sprint 2 output	What it is
Corpus	Kaggle `arshkon/linkedin-job-postings` , 123,849 US postings (Mar–Apr 2024). Duplicates and short texts are tagged, not dropped (106,909 usable). Plus a live Greenhouse feed (453 postings).
Gold set	100 hand-labelled postings across 8 sectors, 1,333 gold skills, split 50 dev / 50 test. Labelling rules A–E; rule E makes ESCO + O*NET the rule book for skill vs trait.
Extractor	**Qwen3.8 27B on Groq (free), prompt v3.** Test F1 strict .66 / lenient .74, a tie with Claude Haiku 5.5 (.66 / .75), so the free model is the pipeline model. Haiku was a research benchmark only.
Pipeline	Postings → clean / dedupe → LLM extract (LiteLLM router, Gemini fallback, Langfuse tracing, every call cached) → ESCO v1.2 / O*NET 31.0 link → Supabase.
Statements	Skill statements in Supabase with an ESCO / O*NET verdict: 69% known skill · 2% trait · 29% new candidate (sent to human review).

What Sprint 3 had to start from: a list of skill *statements*, but no structure. "Excel", "microsoft excel" and "advanced excel" were three unrelated strings, and nothing said which skills belong together. Sprint 2's final report: `reports/sprint2_final_report.md` .

2. What Sprint 3 is, and how we did it

The brief calls Sprint 3 **Representation & Clustering**: "section statements into leaf topics". Its tasks: choose a representation and store it in the shared database, fit and compare at least two clustering methods, run a parameter sweep with a stability report, and decide whether a statement may belong to more than one topic. Deliverable: topic and topic-membership tables plus the stability report with chosen parameters.

Our steps, in order

Step	What we did	Script
1. Snapshot	Froze the statements on 2026-10-22: 28,777 mentions from 1,931 postings, 11,449 distinct skill names. One model per posting (Qwen where its batch is done, the Haiku research run elsewhere; both score the same on test).	`prepare_skills.py`
2. Clean the names	Merged spelling variants and verified aliases: 11,449 → **11,093** skills (details below).	`normalize_aliases.py`, `find_alias_candidates.py`
3. Represent	Embedded each skill name with **BAAI/bge-small-en-v1.5** (384 numbers, pinned revision). Stored in Supabase with pgvector.	`embed_skills.py`, `load_embeddings.py`
4. Cluster + sweep	Tried 4 methods and 20 settings (sections 3 and 5).	`cluster_skills.py`
5. Stability	Re-ran the best settings on random 80% subsets and other seeds.	`cluster_stability.py`
6. Membership	One main topic per skill; noise skills become weak members of the nearest topic; top-3 related skills.	`load_taxonomy.py`
7. Skill library	Generic cross-sector skills get a library label and links to domain topics (section 4).	`build_library.py`, `skill_types.py`
8. Load + check	Topic, membership, library and embedding tables in Supabase; `verify_db.py` 10 pass.	`load_taxonomy.py`, `verify_db.py`

Step 2 in detail: cleaning the skill names

Each name becomes its own point in the embedding space, so "excel" and "microsoft excel" would split counts and blur topics. Four rules merge names: R1 hyphens and punctuation (problem-solving = problem solving), R2 "ms" → "microsoft", R3 drop a trailing "skills", R4 a human-verified alias list.

The R4 list came from a cross-check with independent signals: spelling rules (plurals, spaces, " (ap)"), a level or vendor word ("adobe photoshop" = "photoshop"), near-identical embeddings, and the same ESCO / O*NET entry. All 289 high and medium candidates were checked against the posting text: **269 merged, 16 kept apart**. Kept apart, for example: canvas (Canvas LMS) ≠ canva, "SAPs" (lab analysis plans) ≠ SAP, routings (manufacturing) ≠ routing (networks), CMM (measuring machine) ≠ CMMS. Pairs found only by meaning were usually broad vs narrow ("trend analysis" vs "market trend analysis") and were not merged.

> **Weakness found:** the embedding sees only the name. numpy lands next to "basic math", tensorflow next to airflow ("-flow"), and "microsoft excel" falls between the Microsoft products topic and the spreadsheet topic. Planned fix in Sprint 4: embed the name plus a short description.

3. Clustering methods we tried, with results

Method	How it groups skills	Best setting	Topics	Coverage	AMI	Verdict
HDBSCAN	**finds dense areas in a 10-dim UMAP space; leaves sparse skills as noise**	**size 10, min_samples 1**	**374**	**78%**	**0.84**	chosen
Leiden (RB modularity)	builds a graph of each skill's 15 nearest skills, finds communities	resolution 20	323	100%	0.75	runner-up
Leiden (CPM)	same graph, another quality function (8 settings)	—	148–441	100%	≤ 0.78	rejected
Agglomerative (cosine)	merges the closest pairs step by step, cut at a distance	cut 0.4	357	100%	0.61	hierarchy only
k-means	splits into k round groups	—	–	100%	–	not kept

- **HDBSCAN** gives the steadiest topics on the skills it clusters, and honestly marks 22% as "does not fit". We keep those skills as weak members, so none is lost.
- **Leiden RB** covers every skill and has the best per-topic overlap (Jaccard 0.60), but its labels move more between subsets (AMI 0.75). On the 78% that HDBSCAN clusters, the two agree (AMI 0.78, uncleaned run), so the choice is mainly about the hard 22%.

- **Leiden CPM** was suggested by a GPT review as "steadier as data grows". Our test said no: best per-topic Jaccard 0.53 vs 0.59 for RB.
- **Agglomerative** is the least stable on raw skills and jumps from thousands of tiny clusters to one giant cluster within a small change of the cut. We keep it only for the Sprint 4 hierarchy over topic centres.
- **k-means** needs the number of topics in advance and cannot leave a skill out, which does not fit skill data (topics range from 10 to 90+ skills).

AI advice we tested instead of trusting

GPT review suggested	Result
Leiden with CPM is steadier	rejected CPM Jaccard 0.53 vs RB 0.59
Merge verified aliases first	done 269 evidence-checked merges
Never force a skill into a topic: assigned / weak	done with score and margin per skill
Build the hierarchy over topic centroids	agreed Sprint 4
Give soft skills their own family	done differently the skill library

4. Our skill library work

Why we built a skill library

Clustering groups skills by what they *mean*, using only the skill name. It never looks at which jobs use a skill. That leaves a gap for the generic skills that every sector asks for. We built the library for four reasons:

Reason	What happens without it	What the library adds
1. Generic skills get stranded	HDBSCAN puts many of them in noise (attention to detail has no topic at all) or in separate soft-skill topics (39 topics), cut off from the domains that use them	Each generic skill is linked to the domain topics it really appears with
2. Our end goal needs them	For an initiative like "build a finance team" we could return accounting topics, but not that the same postings also ask for excel and attention to detail	Links found in real postings: microsoft excel → payment reconciliation (lift 3.2)
3. The brief asks for it	"Decide whether a statement may belong to more than one topic" would be answered with one topic per skill only	A library skill belongs to its own topic and links to several domain topics: our multi-topic answer
4. It prepares Sprint 5	Task ↔ skill mapping would start from zero	The same co-occurrence maths (lift, p, q per template) will link Section A's tasks to our skills

Example: communication appears in 603 postings across all 8 sectors. Its meaning puts it in a soft-skill topic, which tells us nothing about where it matters most.

The idea. Keep two things apart: topics (meaning, from embeddings) and a library of cross-sector skills linked to the domain topics they really go with (from counting postings). The library is a label on a skill, so nothing is duplicated.

Who is in the library

- A skill seen in ≥ 5 postings, in ≥ 4 of 8 sectors, with no sector holding more than 50% of its postings. Confirmed = ≥ 10 postings. The rule was fixed before looking at the results.
- **196 library skills** (146 confirmed). Only 43 of them are HDBSCAN noise; the rest also have a normal topic.
- Each has a **type** from its ESCO / O*NET link: 80 skill · 55 knowledge · 27 tool (excel, word) · 6 trait (attention to detail, leadership) · 28 unknown.
- 39 soft-skill topics (more than half their mentions are library skills) are flagged *transversal*.

How a library skill links to a topic

1. Count job templates (company + job title) that mention the skill, the topic, and both.
2. **Lift** = observed ÷ expected by chance. Example: excel 191 postings, the accounts-payable topic 41, of 1,931 → chance ≈ 4 shared, seen 18, lift ≈ 4.4.
3. **p** = hypergeometric test (could the overlap be luck?); **q** = Benjamini–Hochberg correction over all 65,584 pairs tested.
4. **Strong** when ≥ 5 templates, lift ≥ 1.5 and q < 0.05; otherwise weak. Result: **980 links, 249 strong** (e.g. microsoft excel → payment reconciliation, lift 3.2).

How the library improved (v1 → v2)

Change	Why	Effect
Name clean-up before building	excel / microsoft excel were counted apart	microsoft excel now one skill, 250 postings
Count per job template, not per posting	one job copied to many cities counted many times	255 → 249 strong; empathy (7 postings = 4 jobs) and writing lost strong links
Per whole company (tried)	—	rejected staffing firms post unrelated roles; 255 → 154 for the wrong reason
Skill type added	tools, traits and skills were mixed	traits can be filtered (rule E); tools are visible

Study page with interactive examples: `learn/library-links.html`. In the UI: Skill map → click a library skill to see its links.

5. How we did the parameter sweep

Grid. All runs on the final 11,093 skills, seed 42: HDBSCAN min cluster size 5 / 10 / 15 / 20 × min_samples default / 1 / 3 (12 settings); Leiden resolution 5 / 10 / 20 / 30 (k = 15); agglomerative cut 0.30 / 0.35 / 0.40 / 0.45. Per run we record topics, noise share, median and largest topic size, silhouette.

Method	Setting	Topics	Noise	Median size	Largest
HDBSCAN	size 5, default	586	27.0%	10	75
HDBSCAN	size 10, default	275	37.4%	20	132
HDBSCAN	size 15, default	163	42.7%	31	445
HDBSCAN	size 20, default	113	47.5%	40	411
HDBSCAN	size 5, min_samples 1	752	19.3%	9	78
HDBSCAN	**size 10, min_samples 1 (chosen)**	**374**	**22.1%**	**18.5**	**91**
HDBSCAN	size 15, min_samples 1	244	24.3%	27	202
HDBSCAN	size 20, min_samples 1	184	27.0%	36	202
HDBSCAN	size 5, min_samples 3	691	21.2%	9	78
HDBSCAN	size 10, min_samples 3	363	24.0%	19	134
HDBSCAN	size 15, min_samples 3	236	26.5%	27	201
HDBSCAN	size 20, min_samples 3	177	28.2%	36	201
Leiden	resolution 5	116	0%	92	273
Leiden	resolution 10	212	0%	49	143
Leiden	resolution 20	323	0%	31	101
Leiden	resolution 30	412	0%	25	78
Agglomerative	cut 0.30	1,975	0%	3	98
Agglomerative	cut 0.35	1,012	0%	4	206
Agglomerative	cut 0.40	357	0%	8	814
Agglomerative	cut 0.45	67	0%	11	5,028

Reading the sweep: bigger HDBSCAN sizes push more skills into noise and create one oversized topic; size 5 makes hundreds of tiny topics; min_samples 1 keeps noise near 20%. An earlier sweep on the uncleaned names showed the same pattern.

Stability. For the strongest settings we re-ran clustering on 5 random 80% subsets (only the data changes) and with 3 other seeds (only the seed changes). **AMI** compares labels between

runs (1 = identical, 0 = chance). **Per-topic Jaccard** (Hennig 2007) is each topic's best overlap with any subset cluster: ≥ 0.75 stable, < 0.5 dissolved.

Setting	Topics	AMI subsets	AMI seeds	Topic Jaccard	Dissolved	Coverage
HDBSCAN 10, default	275	0.908	0.974	0.580	42%	63%
HDBSCAN 10, min_samples 3	363	0.853	0.952	0.538	46%	76%
HDBSCAN 10, min_samples 1	**374**	**0.842**	**0.946**	**0.535**	**45%**	**78%**
HDBSCAN 15, min_samples 1	244	0.838	0.942	0.498	50%	76%
Leiden RB res 20	323	0.746	0.868	0.596	36%	100%
Agglomerative 0.4	357	0.606	1.000	0.452	57%	100%

- "HDBSCAN 10 default" scores highest only because it leaves 37% of skills out (HDBSCAN AMI is measured on clustered skills).
- min_samples 3 is a near-tie (0.853 vs 0.842) with 2 points less coverage; we keep min_samples 1.
- About half of the topics dissolve in some subset for every method, so topic-level claims need human review.

6. Final result of Sprint 3

> **Chosen:** HDBSCAN, `min_cluster_size = 10`, `min_samples = 1`, on UMAP (10 dims, 15 neighbours, min_dist 0, cosine, seed 42) of bge-small-en-v1.5 embeddings of 11,093 clean skill names. Noise skills are weak members of the nearest topic. Plus skill library v2.

Result	Value
Clean skills (vector points)	11,093
Leaf topics	374
Skills in a topic / weak members	8,646 / 2,447
Stability: AMI subsets / seeds / per-topic Jaccard	0.84 / 0.95 / 0.53
Library skills (confirmed) · transversal topics	196 (146) · 39
Library links (strong)	980 (249)
Embeddings in Supabase (pgvector)	11,093 × 384

Brief checklist

Sprint 3 task	Status
Representation chosen and stored in the shared database	done `skill_embeddings`
At least two clustering methods compared	done four
Parameter sweep and stability report	done this page
Multi-topic decision implemented	done main topic + weak + library links
Topic and topic-membership tables	done `topics`, `skill_topics` (run `hdbscan_10_umap10_ms1_2026-10-23`)

Limits to say out loud

- About half the topics are fragile under resampling, for every method.
- The embedding uses the name only; topic labels are still the centroid skill (the coding topic shows "scala").
- Small sample (1,931 postings); the Qwen batch is slowed by the free tier, so the snapshot mixes Qwen and the Haiku research run.
- Library thresholds were set by us and are not yet tested for sensitivity; sectors come from noisy LinkedIn tags.

Next

- **Sprint 4:** name every topic, build parent levels over topic centres, keep the library as a cross-sector branch (as ESCO does), evaluate against ESCO.
- **Sprint 5 (with Section A):** task ↔ skill mapping; needs shared posting ids (data contract).

Appendix: reproduce

`D = data/processed/sprint3_full_2026-10-22`, `V = D/canonical_v2`; Python 3.11.

```
python src/normalize_aliases.py --mentions D/prepared_mentions.csv --skills D/unique_skills
python src/embed_skills.py --skills V/unique_skills.csv --model-config V/embedding_model.js
python src/cluster_skills.py --emb-dir V/embeddings --out-dir V/clusters --method hdbscan -
python src/cluster_stability.py --emb-dir V/embeddings --out V/stability.csv --method hdbsc
python src/skill_types.py --skills V/unique_skills.csv --out V/skill_types.csv
python src/build_library.py --mentions D/prepared_mentions.csv --aliases V/aliases.csv \
    --run V/clusters/hdbscan_10_umap10_ms1 --out-dir V/library_hdbscan_v2 --types V/skill_t
python src/load_taxonomy.py --run V/clusters/hdbscan_10_umap10_ms1 --library V/library_hdbs
    --emb-dir V/embeddings --aliases V/aliases.csv --activate
```

```
python src/load_embeddings.py --emb-dir V/embeddings
python src/verify_db.py
```

Slide text with speaker notes: `docs/sprint3_demo_slides.md`. Study pages: `learn/cluster-topics.html`, `learn/library-links.html`.